

All-Pairs Shortest Paths Algorithms

Dr. Bugra Caskurlu¹

¹ Artificial Intelligence Engineering Department
TOBB University of Economics and Technology

February 5, 2024

All-Pairs Shortest Paths Problem

Problem Definition and an Application

- In the **all-pairs shortest paths** problem, the goal is to find, for every pair of vertices $u, v \in V$, a shortest (least-weight) path from u to v , where the weight of a path is the sum of the weights of its constituent edges.

All-Pairs Shortest Paths Problem

Problem Definition and an Application

- In the **all-pairs shortest paths** problem, the goal is to find, for every pair of vertices $u, v \in V$, a shortest (least-weight) path from u to v , where the weight of a path is the sum of the weights of its constituent edges.
- The input is a weighted, directed graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$ that maps edges to real-valued weights. The edges may have negative weights, but we assume that there is no negative-weight cycles.

All-Pairs Shortest Paths Problem

Problem Definition and an Application

- In the **all-pairs shortest paths** problem, the goal is to find, for every pair of vertices $u, v \in V$, a shortest (least-weight) path from u to v , where the weight of a path is the sum of the weights of its constituent edges.
- The input is a weighted, directed graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$ that maps edges to real-valued weights. The edges may have negative weights, but we assume that there is no negative-weight cycles.
- The output typically takes a tabular form in which the entry in u 's row and v 's column is the weight of a shortest path from u to v .

All-Pairs Shortest Paths Problem

Problem Definition and an Application

- In the **all-pairs shortest paths** problem, the goal is to find, for every pair of vertices $u, v \in V$, a shortest (least-weight) path from u to v , where the weight of a path is the sum of the weights of its constituent edges.
- The input is a weighted, directed graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$ that maps edges to real-valued weights. The edges may have negative weights, but we assume that there is no negative-weight cycles.
- The output typically takes a tabular form in which the entry in u 's row and v 's column is the weight of a shortest path from u to v .
- An application of all-pairs shortest paths is to determine the **diameter** of a network: the longest of all shortest paths.

All-Pairs Shortest Paths Problem

Problem Definition and an Application

- In the **all-pairs shortest paths** problem, the goal is to find, for every pair of vertices $u, v \in V$, a shortest (least-weight) path from u to v , where the weight of a path is the sum of the weights of its constituent edges.
- The input is a weighted, directed graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$ that maps edges to real-valued weights. The edges may have negative weights, but we assume that there is no negative-weight cycles.
- The output typically takes a tabular form in which the entry in u 's row and v 's column is the weight of a shortest path from u to v .
- An application of all-pairs shortest paths is to determine the **diameter** of a network: the longest of all shortest paths.
- If a directed graph models a communication network, with the weight of an edge indicating the time required for a message to traverse a communication link, then the diameter gives the longest possible transit time for a message in the network.

Naive Algorithms To Solve All-Pairs Shortest Paths Problem

Using Single-Source Shortest Path Algorithms

- You can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once with each vertex as the source.

Naive Algorithms To Solve All-Pairs Shortest Paths Problem

Using Single-Source Shortest Path Algorithms

- You can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once with each vertex as the source.
- If all edge weights are non-negative, you can use Dijkstra's algorithm. If you implement the min-priority queue with a linear array, the running time is $O(n^3 + n \cdot m)$ which is $O(n^3)$.

Naive Algorithms To Solve All-Pairs Shortest Paths Problem

Using Single-Source Shortest Path Algorithms

- You can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once with each vertex as the source.
- If all edge weights are non-negative, you can use Dijkstra's algorithm. If you implement the min-priority queue with a linear array, the running time is $O(n^3 + n \cdot m)$ which is $O(n^3)$.
- The binary min-heap implementation of the min-priority queue yields a running time of $O(n \cdot m \cdot \lg n)$, which is faster than $O(n^3)$ if the graph is sparse.

Naive Algorithms To Solve All-Pairs Shortest Paths Problem

Using Single-Source Shortest Path Algorithms

- You can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once with each vertex as the source.
- If all edge weights are non-negative, you can use Dijkstra's algorithm. If you implement the min-priority queue with a linear array, the running time is $O(n^3 + n \cdot m)$ which is $O(n^3)$.
- The binary min-heap implementation of the min-priority queue yields a running time of $O(n \cdot m \cdot \lg n)$, which is faster than $O(n^3)$ if the graph is sparse.
- If the graph contains negative-weight edges, Dijkstra's algorithm doesn't work, but you can run the slower Bellman-Ford algorithm once from each vertex.

Naive Algorithms To Solve All-Pairs Shortest Paths Problem

Using Single-Source Shortest Path Algorithms

- You can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once with each vertex as the source.
- If all edge weights are non-negative, you can use Dijkstra's algorithm. If you implement the min-priority queue with a linear array, the running time is $O(n^3 + n \cdot m)$ which is $O(n^3)$.
- The binary min-heap implementation of the min-priority queue yields a running time of $O(n \cdot m \cdot \lg n)$, which is faster than $O(n^3)$ if the graph is sparse.
- If the graph contains negative-weight edges, Dijkstra's algorithm doesn't work, but you can run the slower Bellman-Ford algorithm once from each vertex.
- The resulting running time is $O(n^2 \cdot m)$, which on a dense graph is $O(n^4)$.

Naive Algorithms To Solve All-Pairs Shortest Paths Problem

Using Single-Source Shortest Path Algorithms

- You can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once with each vertex as the source.
- If all edge weights are non-negative, you can use Dijkstra's algorithm. If you implement the min-priority queue with a linear array, the running time is $O(n^3 + n \cdot m)$ which is $O(n^3)$.
- The binary min-heap implementation of the min-priority queue yields a running time of $O(n \cdot m \cdot \lg n)$, which is faster than $O(n^3)$ if the graph is sparse.
- If the graph contains negative-weight edges, Dijkstra's algorithm doesn't work, but you can run the slower Bellman-Ford algorithm once from each vertex.
- The resulting running time is $O(n^2 \cdot m)$, which on a dense graph is $O(n^4)$.
- Today, we will see how to guarantee a much better asymptotic running time. We will also investigate the relation of the all-pairs shortest-paths problem to matrix multiplication.

Input and Output Format

Specifying Input and Output Matrices

- Unlike the single-source algorithms, which assume an adjacency-list representation of the graph, the algorithms we will see today (except Johnson's algorithm for sparse graphs) represent the graph by an adjacency matrix.

Input and Output Format

Specifying Input and Output Matrices

- Unlike the single-source algorithms, which assume an adjacency-list representation of the graph, the algorithms we will see today (except Johnson's algorithm for sparse graphs) represent the graph by an adjacency matrix.
- For convenience, we assume that the vertices are numbered $1, 2, \dots, |V|$, so that the input is an $n \times n$ matrix $W = (w_{ij})$ representing the edge weights of a directed graph $G = (V, E)$ with n vertices, where

$$w_{ij} = \begin{cases} 0 & \text{if } i = j, \\ \text{the weight of directed edge } (i, j) & \text{if } i \neq j \text{ and } (i, j) \in E, \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E. \end{cases}$$

Input and Output Format

Specifying Input and Output Matrices

- Unlike the single-source algorithms, which assume an adjacency-list representation of the graph, the algorithms we will see today (except Johnson's algorithm for sparse graphs) represent the graph by an adjacency matrix.
- For convenience, we assume that the vertices are numbered $1, 2, \dots, |V|$, so that the input is an $n \times n$ matrix $W = (w_{ij})$ representing the edge weights of a directed graph $G = (V, E)$ with n vertices, where

$$w_{ij} = \begin{cases} 0 & \text{if } i = j, \\ \text{the weight of directed edge } (i, j) & \text{if } i \neq j \text{ and } (i, j) \in E, \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E. \end{cases}$$

- The output of each of our all-pairs shortest-paths algorithms is an $n \times n$ matrix. The (i, j) entry of the output matrix contains $\delta(i, j)$, the shortest-path weight from vertex i to vertex j .

Predecessor Matrix

Specifying Predecessor Matrix

- A full solution to the all-pairs shortest-paths problem includes not only the shortest-path weights but also a **predecessor matrix** $\Pi = (\pi_{ij})$, where π_{ij} is NIL if either $i = j$ or there is no path from i to j , and otherwise π_{ij} is the predecessor of j on some shortest path from i .

Predecessor Matrix

Specifying Predecessor Matrix

- A full solution to the all-pairs shortest-paths problem includes not only the shortest-path weights but also a **predecessor matrix** $\Pi = (\pi_{ij})$, where π_{ij} is NIL if either $i = j$ or there is no path from i to j , and otherwise π_{ij} is the predecessor of j on some shortest path from i .
- Just as the predecessor subgraph G_π is a shortest-paths tree for a given source vertex, the subgraph induced by the i^{th} row of the Π matrix should be a shortest-paths tree with root i .

Predecessor Matrix

Specifying Predecessor Matrix

- A full solution to the all-pairs shortest-paths problem includes not only the shortest-path weights but also a **predecessor matrix** $\Pi = (\pi_{ij})$, where π_{ij} is NIL if either $i = j$ or there is no path from i to j , and otherwise π_{ij} is the predecessor of j on some shortest path from i .
- Just as the predecessor subgraph G_π is a shortest-paths tree for a given source vertex, the subgraph induced by the i^{th} row of the Π matrix should be a shortest-paths tree with root i .
- For each vertex $i \in V$, the **predecessor subgraph** of G for i is $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$, where $V_{\pi,i} = \{j \in V : \pi_{ij} \neq \text{NIL} \cup \{i\}\}$, $E_{\pi,i} = \{(\pi_{ij}, j) : j \in V_{\pi,i} \setminus \{i\}\}$.

Predecessor Matrix

Specifying Predecessor Matrix

- A full solution to the all-pairs shortest-paths problem includes not only the shortest-path weights but also a **predecessor matrix** $\Pi = (\pi_{ij})$, where π_{ij} is NIL if either $i = j$ or there is no path from i to j , and otherwise π_{ij} is the predecessor of j on some shortest path from i .
- Just as the predecessor subgraph G_π is a shortest-paths tree for a given source vertex, the subgraph induced by the i^{th} row of the Π matrix should be a shortest-paths tree with root i .
- For each vertex $i \in V$, the **predecessor subgraph** of G for i is $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$, where $V_{\pi,i} = \{j \in V : \pi_{ij} \neq \text{NIL} \cup \{i\}\}$, $E_{\pi,i} = \{(\pi_{ij}, j) : j \in V_{\pi,i} \setminus \{i\}\}$.

PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, j)

```

1  if  $i == j$ 
2      print  $i$ 
3  elseif  $\pi_{ij} == \text{NIL}$ 
4      print "no path from"  $i$  "to"  $j$  "exists"
5  else PRINT-ALL-PAIRS-SHORTEST-PATH( $\Pi, i, \pi_{ij}$ )
6      print  $j$ 
```

Figure: If $G_{\pi,i}$ is a shortest-paths tree, this procedure prints a shortest path from vertex i to vertex j .

A Recursive Solution Using The Structure Of Shortest Paths

All Subpaths of a Shortest Path are Shortest Paths

Consider a shortest path p from vertex i to vertex j , and suppose that p contains at most r edges. Assuming that there are no negative-weight cycles, r is finite. If $i = j$, then p has weight 0 and no edges. If vertices i and j are distinct, then decompose path p into $i \xrightarrow{p'} k \rightarrow j$, where path p' now contains at most $r - 1$ edges. p' is a shortest path from i to k , and so $\delta(i, j) = \delta(i, k) + w_{kj}$.

A Recursive Solution Using The Structure Of Shortest Paths

All Subpaths of a Shortest Path are Shortest Paths

Consider a shortest path p from vertex i to vertex j , and suppose that p contains at most r edges. Assuming that there are no negative-weight cycles, r is finite. If $i = j$, then p has weight 0 and no edges. If vertices i and j are distinct, then decompose path p into $i \xrightarrow{p'} k \rightarrow j$, where path p' now contains at most $r - 1$ edges. p' is a shortest path from i to k , and so $\delta(i, j) = \delta(i, k) + w_{kj}$.

A Recursive Structure for Shortest Paths

- Let $\ell_{ij}^{(r)}$ be the minimum weight of any path from i to j that contains at most r edges.

A Recursive Solution Using The Structure Of Shortest Paths

All Subpaths of a Shortest Path are Shortest Paths

Consider a shortest path p from vertex i to vertex j , and suppose that p contains at most r edges. Assuming that there are no negative-weight cycles, r is finite. If $i = j$, then p has weight 0 and no edges. If vertices i and j are distinct, then decompose path p into $i \xrightarrow{p'} k \rightarrow j$, where path p' now contains at most $r - 1$ edges. p' is a shortest path from i to k , and so $\delta(i, j) = \delta(i, k) + w_{kj}$.

A Recursive Structure for Shortest Paths

- Let $l_{ij}^{(r)}$ be the minimum weight of any path from i to j that contains at most r edges.
- When $r = 0$, there is a shortest path from i to j with no edges if and only if $i = j$, yielding

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j, \\ \infty & \text{if } i \neq j. \end{cases}$$

A Recursive Solution Using The Structure Of Shortest Paths

All Subpaths of a Shortest Path are Shortest Paths

Consider a shortest path p from vertex i to vertex j , and suppose that p contains at most r edges. Assuming that there are no negative-weight cycles, r is finite. If $i = j$, then p has weight 0 and no edges. If vertices i and j are distinct, then decompose path p into $i \xrightarrow{p'} k \rightarrow j$, where path p' now contains at most $r - 1$ edges. p' is a shortest path from i to k , and so $\delta(i, j) = \delta(i, k) + w_{kj}$.

A Recursive Structure for Shortest Paths

- Let $l_{ij}^{(r)}$ be the minimum weight of any path from i to j that contains at most r edges.
- When $r = 0$, there is a shortest path from i to j with no edges if and only if $i = j$, yielding

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j, \\ \infty & \text{if } i \neq j. \end{cases}$$

- For $r \geq 1$, one way to achieve a minimum-weight path from i to j with at most r edges is by taking a path containing at most $r - 1$ edges, so that $l_{ij}^{(r)} = l_{ij}^{(r-1)}$.

A Recursive Solution Using The Structure Of Shortest Paths

All Subpaths of a Shortest Path are Shortest Paths

Consider a shortest path p from vertex i to vertex j , and suppose that p contains at most r edges. Assuming that there are no negative-weight cycles, r is finite. If $i = j$, then p has weight 0 and no edges. If vertices i and j are distinct, then decompose path p into $i \xrightarrow{p'} k \rightarrow j$, where path p' now contains at most $r - 1$ edges. p' is a shortest path from i to k , and so $\delta(i, j) = \delta(i, k) + w_{kj}$.

A Recursive Structure for Shortest Paths

- Let $l_{ij}^{(r)}$ be the minimum weight of any path from i to j that contains at most r edges.
- When $r = 0$, there is a shortest path from i to j with no edges if and only if $i = j$, yielding

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j, \\ \infty & \text{if } i \neq j. \end{cases}$$

- For $r \geq 1$, one way to achieve a minimum-weight path from i to j with at most r edges is by taking a path containing at most $r - 1$ edges, so that $l_{ij}^{(r)} = l_{ij}^{(r-1)}$. Another way is by taking a path of at most $r - 1$ edges from i to some vertex k and then taking the edge (k, j) , so that $l_{ij}^{(r)} = l_{ik}^{(r-1)} + w(k, j)$.

Extending Shortest Paths

Obtaining A Recursive Formulation

- To examine paths from i to j consisting of at most r edges, trying all possible predecessors k of j , gives the recursive definition

$$l_{ij}^{(r)} = \min \left\{ l_{ij}^{(r-1)}, \min \{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \} \right\} \quad (1)$$

$$= \min \{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \}. \text{ (since } w_{jj} = 0 \text{ for all } j. \text{)} \quad (2)$$

Extending Shortest Paths

Obtaining A Recursive Formulation

- To examine paths from i to j consisting of at most r edges, trying all possible predecessors k of j , gives the recursive definition

$$l_{ij}^{(r)} = \min \left\{ l_{ij}^{(r-1)}, \min \{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \} \right\} \quad (1)$$

$$= \min \{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \}. \text{ (since } w_{jj} = 0 \text{ for all } j.) \quad (2)$$

- Given $L^{(r-1)} = \left(l_{ij}^{(r-1)} \right)$, and the edge-weight matrix W , we can compute $L^{(r)} = \left(l_{ij}^{(r)} \right)$, whose elements are initialized to ∞ in $\Theta(n^3)$ -time by the following pseudocode.

Extending Shortest Paths

Obtaining A Recursive Formulation

- To examine paths from i to j consisting of at most r edges, trying all possible predecessors k of j , gives the recursive definition

$$l_{ij}^{(r)} = \min \left\{ l_{ij}^{(r-1)}, \min \{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \} \right\} \quad (1)$$

$$= \min \{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \}. \text{ (since } w_{jj} = 0 \text{ for all } j.) \quad (2)$$

- Given $L^{(r-1)} = (l_{ij}^{(r-1)})$, and the edge-weight matrix W , we can compute $L^{(r)} = (l_{ij}^{(r)})$, whose elements are initialized to ∞ in $\Theta(n^3)$ -time by the following pseudocode.

EXTEND-SHORTEST-PATHS($L^{(r-1)}, W, L^{(r)}, n$)

```

1  // Assume that the elements of  $L^{(r)}$  are initialized to  $\infty$ .
2  for  $i = 1$  to  $n$ 
3      for  $j = 1$  to  $n$ 
4          for  $k = 1$  to  $n$ 
5               $l_{ij}^{(r)} = \min \{ l_{ij}^{(r)}, l_{ik}^{(r-1)} + w_{kj} \}$ 
```

Relating All-Pairs Shortest Paths With Matrix Multiplication

Resemblance Between EXTEND-SHORTEST-PATHS and Matrix Multiplication

- Let's now understand the relation of this computation to matrix multiplication.

Relating All-Pairs Shortest Paths With Matrix Multiplication

Resemblance Between EXTEND-SHORTEST-PATHS and Matrix Multiplication

- Let's now understand the relation of this computation to matrix multiplication.
- Consider how to compute the matrix product $C = A \cdot B$ of two $n \times n$ matrices A and B .

Relating All-Pairs Shortest Paths With Matrix Multiplication

Resemblance Between EXTEND-SHORTEST-PATHS and Matrix Multiplication

- Let's now understand the relation of this computation to matrix multiplication.
- Consider how to compute the matrix product $C = A \cdot B$ of two $n \times n$ matrices A and B .
- The straightforward method uses a triply nested loop to implement the equation

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj},$$

for $i, j = 1, 2, \dots, n$.

Relating All-Pairs Shortest Paths With Matrix Multiplication

Resemblance Between EXTEND-SHORTEST-PATHS and Matrix Multiplication

- Let's now understand the relation of this computation to matrix multiplication.
- Consider how to compute the matrix product $C = A \cdot B$ of two $n \times n$ matrices A and B .
- The straightforward method uses a triply nested loop to implement the equation

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj},$$

for $i, j = 1, 2, \dots, n$.

- Consider the following substitutions in our recurrence $l_{ij}^{(r)} = \min\{l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n\}$:

$$l^{(r-1)} \rightarrow a, w \rightarrow b, l^{(r)} \rightarrow c, \min \rightarrow +, \text{ and } + \rightarrow \cdot$$

- Making these changes to EXTEND-SHORTEST-PATHS, and also replacing ∞ (the identity for min) by 0 (the identity for $+$), yields a procedure for matrix multiplication.

An Algorithm To Compute All-Pairs Shortest Paths

Resemblance With Repeated Matrix Multiplication

- Thus, we can solve the all-pairs shortest-paths problem by repeatedly "multiplying" matrices.

An Algorithm To Compute All-Pairs Shortest Paths

Resemblance With Repeated Matrix Multiplication

- Thus, we can solve the all-pairs shortest-paths problem by repeatedly "multiplying" matrices.
- Each step extends the shortest-path weights computed so far by one more edge using $\text{EXTEND-SHORTEST-PATHS}(L^{(r-1)}, W, L^{(r)}, n)$ to perform the matrix multiplication.

An Algorithm To Compute All-Pairs Shortest Paths

Resemblance With Repeated Matrix Multiplication

- Thus, we can solve the all-pairs shortest-paths problem by repeatedly "multiplying" matrices.
- Each step extends the shortest-path weights computed so far by one more edge using $\text{EXTEND-SHORTEST-PATHS}(L^{(r-1)}, W, L^{(r)}, n)$ to perform the matrix multiplication.
- Starting with the matrix $L^{(0)}$, we produce the following sequence of $n - 1$ matrices corresponding to powers of W :

$$\begin{aligned} L^{(1)} &= L^{(0)} \cdot W = W^1, \\ L^{(2)} &= L^{(1)} \cdot W = W^2, \\ L^{(3)} &= L^{(2)} \cdot W = W^3, \\ &\vdots \\ L^{(n-1)} &= L^{(n-2)} \cdot W = W^{n-1}. \end{aligned}$$

An Algorithm To Compute All-Pairs Shortest Paths

Resemblance With Repeated Matrix Multiplication

- Thus, we can solve the all-pairs shortest-paths problem by repeatedly "multiplying" matrices.
- Each step extends the shortest-path weights computed so far by one more edge using $\text{EXTEND-SHORTEST-PATHS}(L^{(r-1)}, W, L^{(r)}, n)$ to perform the matrix multiplication.
- Starting with the matrix $L^{(0)}$, we produce the following sequence of $n - 1$ matrices corresponding to powers of W :

$$\begin{aligned} L^{(1)} &= L^{(0)} \cdot W = W^1, \\ L^{(2)} &= L^{(1)} \cdot W = W^2, \\ L^{(3)} &= L^{(2)} \cdot W = W^3, \\ &\vdots \\ L^{(n-1)} &= L^{(n-2)} \cdot W = W^{n-1}. \end{aligned}$$

- At the end, the matrix $L^{(n-1)} = W^{n-1}$ contains the shortest-path weights.

Pseudocode of an All-Pairs Shortest Paths Algorithm

```
SLOW-APSP( $W, L^{(0)}, n$ )  
1  let  $L = (l_{ij})$  and  $M = (m_{ij})$  be new  $n \times n$  matrices  
2   $L = L^{(0)}$   
3  for  $r = 1$  to  $n - 1$   
4       $M = \infty$  // initialize  $M$   
5      // Compute the matrix “product”  $M = L \cdot W$ .  
6      EXTEND-SHORTEST-PATHS( $L, W, M, n$ )  
7       $L = M$   
8  return  $L$ 
```

Figure: The $n - 1$ invocations of EXTEND-SHORTEST-PATHS, each of which takes $\Theta(n^3)$ -time, dominate the computation, yielding a total running time of $\Theta(n^4)$.

Improving The Running Time

Repeated Squaring

- Notice that our goal is not to compute all the $L^{(r)}$ matrices: only the matrix $L^{(n-1)}$ matters.

Improving The Running Time

Repeated Squaring

- Notice that our goal is not to compute all the $L^{(r)}$ matrices: only the matrix $L^{(n-1)}$ matters.
- In the absence of negative-weight cycles, $L^{(r)} = L^{(n-1)}$ for all integers $r \geq n - 1$.

Improving The Running Time

Repeated Squaring

- Notice that our goal is not to compute all the $L^{(r)}$ matrices: only the matrix $L^{(n-1)}$ matters.
- In the absence of negative-weight cycles, $L^{(r)} = L^{(n-1)}$ for all integers $r \geq n-1$.
- Just as traditional matrix multiplication is associative, so is matrix multiplication defined by the EXTEND-SHORTEST-PATHS procedure. In fact, we can compute $L^{(n-1)}$ with only $\lceil \lg(n-1) \rceil$ matrix products by using the technique of **repeated squaring**:

$$\begin{array}{rclcl} L^{(1)} & = & L^{(0)} \cdot W & = & W^1, \\ L^{(2)} & = & L^{(1)} \cdot W & = & W^2, \\ L^{(3)} & = & L^{(2)} \cdot W & = & W^3, \\ & & \vdots & & \\ L^{(n-1)} & = & L^{(n-2)} \cdot W & = & W^{n-1}. \end{array}$$

Improving The Running Time

Repeated Squaring

- Notice that our goal is not to compute all the $L^{(r)}$ matrices: only the matrix $L^{(n-1)}$ matters.
- In the absence of negative-weight cycles, $L^{(r)} = L^{(n-1)}$ for all integers $r \geq n-1$.
- Just as traditional matrix multiplication is associative, so is matrix multiplication defined by the EXTEND-SHORTEST-PATHS procedure. In fact, we can compute $L^{(n-1)}$ with only $\lceil \lg(n-1) \rceil$ matrix products by using the technique of **repeated squaring**:

$$\begin{array}{rclcl}
 L^{(1)} & = & L^{(0)} \cdot W & = & W^1, \\
 L^{(2)} & = & L^{(1)} \cdot W & = & W^2, \\
 L^{(3)} & = & L^{(2)} \cdot W & = & W^3, \\
 & & \vdots & & \\
 L^{(n-1)} & = & L^{(n-2)} \cdot W & = & W^{n-1}.
 \end{array}$$

- Since $2^{\lceil \lg(n-1) \rceil} \geq n-1$, the final product is $L^{(2^{\lceil \lg(n-1) \rceil})} = L^{(n-1)}$.

Pseudocode of a Faster All-Pairs Shortest Paths Algorithm

```
FASTER-APSP( $W, n$ )
1  let  $L$  and  $M$  be new  $n \times n$  matrices
2   $L = W$ 
3   $r = 1$ 
4  while  $r < n - 1$ 
5       $M = \infty$     // initialize  $M$ 
6      EXTEND-SHORTEST-PATHS( $L, L, M, n$ )    // compute  $M = L^2$ 
7       $r = 2r$ 
8       $L = M$     // ready for the next iteration
9  return  $L$ 
```

Figure: Because each of the $\lceil \lg(n-1) \rceil$ matrix products takes $\Theta(n^3)$ time, FASTER-APSP runs in $\Theta(n^3 \cdot \lg n)$ time. The code is tight, containing no elaborate data structures, and the constant hidden in the *Theta* notation is therefore small.

The Key Observation of Floyd and Warshall

A Different Characterization of Shortest Paths

- Having already seen one dynamic-programming solution to the all-pairs shortest-paths problem, now we'll see another: the **Floyd-Warshall algorithm**, which runs in $\Theta(n^3)$ time.

The Key Observation of Floyd and Warshall

A Different Characterization of Shortest Paths

- Having already seen one dynamic-programming solution to the all-pairs shortest-paths problem, now we'll see another: the **Floyd-Warshall algorithm**, which runs in $\Theta(n^3)$ time.
- As before, negative-weight edges may be present, but not negative-weight cycles.

The Key Observation of Floyd and Warshall

A Different Characterization of Shortest Paths

- Having already seen one dynamic-programming solution to the all-pairs shortest-paths problem, now we'll see another: the **Floyd-Warshall algorithm**, which runs in $\Theta(n^3)$ time.
- As before, negative-weight edges may be present, but not negative-weight cycles.
- In the Floyd-Warshall algorithm, our characterization of the structure of a shortest path is different.

The Key Observation of Floyd and Warshall

A Different Characterization of Shortest Paths

- Having already seen one dynamic-programming solution to the all-pairs shortest-paths problem, now we'll see another: the **Floyd-Warshall algorithm**, which runs in $\Theta(n^3)$ time.
- As before, negative-weight edges may be present, but not negative-weight cycles.
- In the Floyd-Warshall algorithm, our characterization of the structure of a shortest path is different.
- The Floyd-Warshall algorithm considers the intermediate vertices of a shortest path, where an **intermediate** vertex of a simple path $p = \langle v_1, v_2, \dots, v_l \rangle$ is any vertex of p other than v_1 or v_l , that is, any vertex in the set $\{v_2, v_3, \dots, v_{l-1}\}$.

The Key Observation of Floyd and Warshall

A Different Characterization of Shortest Paths

- Having already seen one dynamic-programming solution to the all-pairs shortest-paths problem, now we'll see another: the **Floyd-Warshall algorithm**, which runs in $\Theta(n^3)$ time.
- As before, negative-weight edges may be present, but not negative-weight cycles.
- In the Floyd-Warshall algorithm, our characterization of the structure of a shortest path is different.
- The Floyd-Warshall algorithm considers the intermediate vertices of a shortest path, where an **intermediate** vertex of a simple path $p = \langle v_1, v_2, \dots, v_l \rangle$ is any vertex of p other than v_1 or v_l , that is, any vertex in the set $\{v_2, v_3, \dots, v_{l-1}\}$.
- The Floyd-Warshall algorithm relies on the following observation: Numbering the vertices of G by $V = \{1, 2, \dots, n\}$, take a subset $\{1, 2, \dots, k\}$ of vertices for some $1 \leq k \leq n$.

The Key Observation of Floyd and Warshall

A Different Characterization of Shortest Paths

- Having already seen one dynamic-programming solution to the all-pairs shortest-paths problem, now we'll see another: the **Floyd-Warshall algorithm**, which runs in $\Theta(n^3)$ time.
- As before, negative-weight edges may be present, but not negative-weight cycles.
- In the Floyd-Warshall algorithm, our characterization of the structure of a shortest path is different.
- The Floyd-Warshall algorithm considers the intermediate vertices of a shortest path, where an **intermediate** vertex of a simple path $p = \langle v_1, v_2, \dots, v_l \rangle$ is any vertex of p other than v_1 or v_l , that is, any vertex in the set $\{v_2, v_3, \dots, v_{l-1}\}$.
- The Floyd-Warshall algorithm relies on the following observation: Numbering the vertices of G by $V = \{1, 2, \dots, n\}$, take a subset $\{1, 2, \dots, k\}$ of vertices for some $1 \leq k \leq n$.
- For any pair of vertices $i, j \in V$, consider all paths from i to j whose intermediate vertices are all drawn from $\{1, 2, \dots, k\}$, and let p be a minimum-weight path from among them. Notice that path p is simple.

The Key Observation of Floyd and Warshall

A Different Characterization of Shortest Paths

- Having already seen one dynamic-programming solution to the all-pairs shortest-paths problem, now we'll see another: the **Floyd-Warshall algorithm**, which runs in $\Theta(n^3)$ time.
- As before, negative-weight edges may be present, but not negative-weight cycles.
- In the Floyd-Warshall algorithm, our characterization of the structure of a shortest path is different.
- The Floyd-Warshall algorithm considers the intermediate vertices of a shortest path, where an **intermediate** vertex of a simple path $p = \langle v_1, v_2, \dots, v_l \rangle$ is any vertex of p other than v_1 or v_l , that is, any vertex in the set $\{v_2, v_3, \dots, v_{l-1}\}$.
- The Floyd-Warshall algorithm relies on the following observation: Numbering the vertices of G by $V = \{1, 2, \dots, n\}$, take a subset $\{1, 2, \dots, k\}$ of vertices for some $1 \leq k \leq n$.
- For any pair of vertices $i, j \in V$, consider all paths from i to j whose intermediate vertices are all drawn from $\{1, 2, \dots, k\}$, and let p be a minimum-weight path from among them. Notice that path p is simple.
- The Floyd-Warshall algorithm exploits a relationship between path p and shortest paths from i to j with all intermediate vertices in the set $\{1, 2, \dots, k-1\}$.

The Characterization of the Structure of Shortest Paths Used by Floyd-Warshall

Details

- The details of the relationship between shortest paths from i to j with intermediate vertices drawn from $\{1, 2, \dots, k\}$ versus $\{1, 2, \dots, k - 1\}$, depend on whether k is an intermediate vertex of path p or not.

The Characterization of the Structure of Shortest Paths Used by Floyd-Warshall

Details

- The details of the relationship between shortest paths from i to j with intermediate vertices drawn from $\{1, 2, \dots, k\}$ versus $\{1, 2, \dots, k - 1\}$, depend on whether k is an intermediate vertex of path p or not.
- If k is not an intermediate vertex of path p , then all intermediate vertices of path p belong to the set $\{1, 2, \dots, k - 1\}$.

The Characterization of the Structure of Shortest Paths Used by Floyd-Warshall

Details

- The details of the relationship between shortest paths from i to j with intermediate vertices drawn from $\{1, 2, \dots, k\}$ versus $\{1, 2, \dots, k - 1\}$, depend on whether k is an intermediate vertex of path p or not.
- If k is not an intermediate vertex of path p , then all intermediate vertices of path p belong to the set $\{1, 2, \dots, k - 1\}$.
- Thus a shortest path from vertex i to vertex j with all intermediate vertices in the set $\{1, 2, \dots, k - 1\}$ is also a shortest path from i to j with all intermediate vertices in the set $\{1, 2, \dots, k\}$.

The Characterization of the Structure of Shortest Paths Used by Floyd-Warshall

Details

- The details of the relationship between shortest paths from i to j with intermediate vertices drawn from $\{1, 2, \dots, k\}$ versus $\{1, 2, \dots, k - 1\}$, depend on whether k is an intermediate vertex of path p or not.
- If k is not an intermediate vertex of path p , then all intermediate vertices of path p belong to the set $\{1, 2, \dots, k - 1\}$.
- Thus a shortest path from vertex i to vertex j with all intermediate vertices in the set $\{1, 2, \dots, k - 1\}$ is also a shortest path from i to j with all intermediate vertices in the set $\{1, 2, \dots, k\}$.
- If k is an intermediate vertex of path p , then decompose p into $i \overset{p_1}{\rightsquigarrow} k \overset{p_2}{\rightsquigarrow} j$.

The Characterization of the Structure of Shortest Paths Used by Floyd-Warshall

Details

- The details of the relationship between shortest paths from i to j with intermediate vertices drawn from $\{1, 2, \dots, k\}$ versus $\{1, 2, \dots, k - 1\}$, depend on whether k is an intermediate vertex of path p or not.
- If k is not an intermediate vertex of path p , then all intermediate vertices of path p belong to the set $\{1, 2, \dots, k - 1\}$.
- Thus a shortest path from vertex i to vertex j with all intermediate vertices in the set $\{1, 2, \dots, k - 1\}$ is also a shortest path from i to j with all intermediate vertices in the set $\{1, 2, \dots, k\}$.
- If k is an intermediate vertex of path p , then decompose p into $i \overset{p_1}{\rightsquigarrow} k \overset{p_2}{\rightsquigarrow} j$.
- Now, p_1 is a shortest path from i to k with all intermediate vertices in the set $\{1, 2, \dots, k - 1\}$. Likewise, p_2 is a shortest path from vertex k to vertex j with all intermediate vertices in the set $\{1, 2, \dots, k - 1\}$.

A Recursive Solution to the All-Pairs Shortest-Paths Problem

Details

- Let us now derive a recursive formulation of shortest-path estimates based on these observations.

A Recursive Solution to the All-Pairs Shortest-Paths Problem

Details

- Let us now derive a recursive formulation of shortest-path estimates based on these observations.
- Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices belong to the set $\{1, 2, \dots, k\}$.

A Recursive Solution to the All-Pairs Shortest-Paths Problem

Details

- Let us now derive a recursive formulation of shortest-path estimates based on these observations.
- Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices belong to the set $\{1, 2, \dots, k\}$.
- When $k = 0$, a path from vertex i to vertex j with no intermediate vertex numbered higher than 0 has no intermediate vertices at all.

A Recursive Solution to the All-Pairs Shortest-Paths Problem

Details

- Let us now derive a recursive formulation of shortest-path estimates based on these observations.
- Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices belong to the set $\{1, 2, \dots, k\}$.
- When $k = 0$, a path from vertex i to vertex j with no intermediate vertex numbered higher than 0 has no intermediate vertices at all.
- Such a path has at most one edge, and hence $d_{ij}^{(0)} = w_{ij}$.

A Recursive Solution to the All-Pairs Shortest-Paths Problem

Details

- Let us now derive a recursive formulation of shortest-path estimates based on these observations.
- Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices belong to the set $\{1, 2, \dots, k\}$.
- When $k = 0$, a path from vertex i to vertex j with no intermediate vertex numbered higher than 0 has no intermediate vertices at all.
- Such a path has at most one edge, and hence $d_{ij}^{(0)} = w_{ij}$.
- We can define $d_{ij}^{(k)}$ recursively as

$$d_{ij}^k = \begin{cases} w_{ij} & \text{if } k = 0, \\ \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} & \text{if } k \geq 1. \end{cases}$$

A Recursive Solution to the All-Pairs Shortest-Paths Problem

Details

- Let us now derive a recursive formulation of shortest-path estimates based on these observations.
- Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices belong to the set $\{1, 2, \dots, k\}$.
- When $k = 0$, a path from vertex i to vertex j with no intermediate vertex numbered higher than 0 has no intermediate vertices at all.
- Such a path has at most one edge, and hence $d_{ij}^{(0)} = w_{ij}$.
- We can define $d_{ij}^{(k)}$ recursively as

$$d_{ij}^k = \begin{cases} w_{ij} & \text{if } k = 0, \\ \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} & \text{if } k \geq 1. \end{cases}$$

- Because for any path, all intermediate vertices belong to the set $\{1, 2, \dots, n\}$, the matrix $D^{(n)} = \left(d_{ij}^{(n)}\right)$ gives the final answer: $d_{ij}^{(n)} = \delta(i, j)$ for all $i, j \in V$.

Pseudocode of the Floyd-Warshall Algorithm

FLOYD-WARSHALL(W, n)

1 $D^{(0)} = W$

2 **for** $k = 1$ **to** n

3 let $D^{(k)} = (d_{ij}^{(k)})$ be a new $n \times n$ matrix

4 **for** $i = 1$ **to** n

5 **for** $j = 1$ **to** n

6 $d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$

7 **return** $D^{(n)}$

Figure: The running time of the Floyd-Warshall algorithm is determined by the triply nested for loops and the algorithm runs in $\Theta(n^3)$ time.

Running Time Of Johnson's Algorithm

We Should Use It For Sparse Graphs

- Johnson's algorithm finds shortest paths between all pairs in $O(n^2 \cdot \lg n + n \cdot m)$ if the min-priority queue is implemented with Fibonacci heaps. If the min-priority queue is implemented with min-heaps, then the running time of Johnson's algorithm is $O(n \cdot m \cdot \lg n)$.

Running Time Of Johnson's Algorithm

We Should Use It For Sparse Graphs

- Johnson's algorithm finds shortest paths between all pairs in $O(n^2 \cdot \lg n + n \cdot m)$ if the min-priority queue is implemented with Fibonacci heaps. If the min-priority queue is implemented with min-heaps, then the running time of Johnson's algorithm is $O(n \cdot m \cdot \lg n)$.
- Notice that Johnson's algorithm is asymptotically faster than the Floyd-Warshall algorithm for sparse graphs.

Running Time Of Johnson's Algorithm

We Should Use It For Sparse Graphs

- Johnson's algorithm finds shortest paths between all pairs in $O(n^2 \cdot \lg n + n \cdot m)$ if the min-priority queue is implemented with Fibonacci heaps. If the min-priority queue is implemented with min-heaps, then the running time of Johnson's algorithm is $O(n \cdot m \cdot \lg n)$.
- Notice that Johnson's algorithm is asymptotically faster than the Floyd-Warshall algorithm for sparse graphs.
- *Fibonacci Heap Implementation*: Its running time is $o(n^3)$, if $m = o(n^2)$.

Running Time Of Johnson's Algorithm

We Should Use It For Sparse Graphs

- Johnson's algorithm finds shortest paths between all pairs in $O(n^2 \cdot \lg n + n \cdot m)$ if the min-priority queue is implemented with Fibonacci heaps. If the min-priority queue is implemented with min-heaps, then the running time of Johnson's algorithm is $O(n \cdot m \cdot \lg n)$.
- Notice that Johnson's algorithm is asymptotically faster than the Floyd-Warshall algorithm for sparse graphs.
- *Fibonacci Heap Implementation*: Its running time is $o(n^3)$, if $m = o(n^2)$.
- *Min-Heap Implementation*: Its running time is $o(n^3)$, if $m = o\left(\frac{n^2}{\lg n}\right)$.

Running Time Of Johnson's Algorithm

We Should Use It For Sparse Graphs

- Johnson's algorithm finds shortest paths between all pairs in $O(n^2 \cdot \lg n + n \cdot m)$ if the min-priority queue is implemented with Fibonacci heaps. If the min-priority queue is implemented with min-heaps, then the running time of Johnson's algorithm is $O(n \cdot m \cdot \lg n)$.
- Notice that Johnson's algorithm is asymptotically faster than the Floyd-Warshall algorithm for sparse graphs.
- *Fibonacci Heap Implementation*: Its running time is $o(n^3)$, if $m = o(n^2)$.
- *Min-Heap Implementation*: Its running time is $o(n^3)$, if $m = o\left(\frac{n^2}{\lg n}\right)$.
- The algorithm either returns a matrix of shortest-path weights for all pairs of vertices or reports that the input graph contains a negative-weight cycle.

Running Time Of Johnson's Algorithm

We Should Use It For Sparse Graphs

- Johnson's algorithm finds shortest paths between all pairs in $O(n^2 \cdot \lg n + n \cdot m)$ if the min-priority queue is implemented with Fibonacci heaps. If the min-priority queue is implemented with min-heaps, then the running time of Johnson's algorithm is $O(n \cdot m \cdot \lg n)$.
- Notice that Johnson's algorithm is asymptotically faster than the Floyd-Warshall algorithm for sparse graphs.
- *Fibonacci Heap Implementation*: Its running time is $o(n^3)$, if $m = o(n^2)$.
- *Min-Heap Implementation*: Its running time is $o(n^3)$, if $m = o\left(\frac{n^2}{\lg n}\right)$.
- The algorithm either returns a matrix of shortest-path weights for all pairs of vertices or reports that the input graph contains a negative-weight cycle.
- It uses both Dijkstra's algorithm and the Bellman-Ford algorithm as subroutines.

The Ideas Used in Johnson's Algorithm

Reweighting Technique

- Johnson's algorithm uses the technique of **reweighting**, which works as follows.

The Ideas Used in Johnson's Algorithm

Reweighting Technique

- Johnson's algorithm uses the technique of **reweighting**, which works as follows.
- If all edge weights w in a graph $G = (V, E)$ are non-negative, Dijkstra's algorithm can find shortest paths between all pairs of vertices by running it once from each vertex.

The Ideas Used in Johnson's Algorithm

Reweighting Technique

- Johnson's algorithm uses the technique of **reweighting**, which works as follows.
- If all edge weights w in a graph $G = (V, E)$ are non-negative, Dijkstra's algorithm can find shortest paths between all pairs of vertices by running it once from each vertex.
- With the Fibonacci-heap min-priority queue, the running time of this all-pairs algorithm is $O(n^2 \cdot \lg n + n \cdot m)$. With binary-heap min-priority queue, the running time of this all-pairs algorithm is $n \cdot m \cdot \lg n$.

The Ideas Used in Johnson's Algorithm

Reweighting Technique

- Johnson's algorithm uses the technique of **reweighting**, which works as follows.
- If all edge weights w in a graph $G = (V, E)$ are non-negative, Dijkstra's algorithm can find shortest paths between all pairs of vertices by running it once from each vertex.
- With the Fibonacci-heap min-priority queue, the running time of this all-pairs algorithm is $O(n^2 \cdot \lg n + n \cdot m)$. With binary-heap min-priority queue, the running time of this all-pairs algorithm is $n \cdot m \cdot \lg n$.
- If G has negative-weight edges but no negative-weight cycles, first compute a new set of non-negative edge weights so that Dijkstra's algorithm applies.

The Ideas Used in Johnson's Algorithm

Reweighting Technique

- Johnson's algorithm uses the technique of **reweighting**, which works as follows.
- If all edge weights w in a graph $G = (V, E)$ are non-negative, Dijkstra's algorithm can find shortest paths between all pairs of vertices by running it once from each vertex.
- With the Fibonacci-heap min-priority queue, the running time of this all-pairs algorithm is $O(n^2 \cdot \lg n + n \cdot m)$. With binary-heap min-priority queue, the running time of this all-pairs algorithm is $n \cdot m \cdot \lg n$.
- If G has negative-weight edges but no negative-weight cycles, first compute a new set of non-negative edge weights so that Dijkstra's algorithm applies.
- The new set of edge weights $\hat{w}$ must satisfy two important properties:

The Ideas Used in Johnson's Algorithm

Reweighting Technique

- Johnson's algorithm uses the technique of **reweighting**, which works as follows.
- If all edge weights w in a graph $G = (V, E)$ are non-negative, Dijkstra's algorithm can find shortest paths between all pairs of vertices by running it once from each vertex.
- With the Fibonacci-heap min-priority queue, the running time of this all-pairs algorithm is $O(n^2 \cdot \lg n + n \cdot m)$. With binary-heap min-priority queue, the running time of this all-pairs algorithm is $n \cdot m \cdot \lg n$.
- If G has negative-weight edges but no negative-weight cycles, first compute a new set of non-negative edge weights so that Dijkstra's algorithm applies.
- The new set of edge weights $\hat{w}$ must satisfy two important properties:
 - 1 For all pairs of vertices $u, v \in V$, a path p is a shortest path from u to v using weight function w if and only if p is also a shortest path from u to v using weight function $\hat{w}$.

The Ideas Used in Johnson's Algorithm

Reweighting Technique

- Johnson's algorithm uses the technique of **reweighting**, which works as follows.
- If all edge weights w in a graph $G = (V, E)$ are non-negative, Dijkstra's algorithm can find shortest paths between all pairs of vertices by running it once from each vertex.
- With the Fibonacci-heap min-priority queue, the running time of this all-pairs algorithm is $O(n^2 \cdot \lg n + n \cdot m)$. With binary-heap min-priority queue, the running time of this all-pairs algorithm is $n \cdot m \cdot \lg n$.
- If G has negative-weight edges but no negative-weight cycles, first compute a new set of non-negative edge weights so that Dijkstra's algorithm applies.
- The new set of edge weights $\hat{w}$ must satisfy two important properties:
 - 1 For all pairs of vertices $u, v \in V$, a path p is a shortest path from u to v using weight function w if and only if p is also a shortest path from u to v using weight function $\hat{w}$.
 - 2 For all edges u, v , the new weight $\hat{w}(u, v)$ is non-negative.

The Ideas Used in Johnson's Algorithm

Reweighting Technique

- Johnson's algorithm uses the technique of **reweighting**, which works as follows.
- If all edge weights w in a graph $G = (V, E)$ are non-negative, Dijkstra's algorithm can find shortest paths between all pairs of vertices by running it once from each vertex.
- With the Fibonacci-heap min-priority queue, the running time of this all-pairs algorithm is $O(n^2 \cdot \lg n + n \cdot m)$. With binary-heap min-priority queue, the running time of this all-pairs algorithm is $n \cdot m \cdot \lg n$.
- If G has negative-weight edges but no negative-weight cycles, first compute a new set of non-negative edge weights so that Dijkstra's algorithm applies.
- The new set of edge weights $\hat{w}$ must satisfy two important properties:
 - 1 For all pairs of vertices $u, v \in V$, a path p is a shortest path from u to v using weight function w if and only if p is also a shortest path from u to v using weight function $\hat{w}$.
 - 2 For all edges u, v , the new weight $\hat{w}(u, v)$ is non-negative.
- As we'll see in a moment, preprocessing G to determine the new weight function $\hat{w}$ takes $O(n \cdot m)$ time.

Preserving Shortest Paths by Reweighting

Lemma: Reweighting Does Not Change Shortest Paths

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $h : V \rightarrow \mathbb{R}$ be any function mapping vertices to real numbers.

Preserving Shortest Paths by Reweighting

Lemma: Reweighting Does Not Change Shortest Paths

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $h : V \rightarrow \mathbb{R}$ be any function mapping vertices to real numbers.

For each edge $(u, v) \in E$, define $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$.

Preserving Shortest Paths by Reweighting

Lemma: Reweighting Does Not Change Shortest Paths

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $h : V \rightarrow \mathbb{R}$ be any function mapping vertices to real numbers.

For each edge $(u, v) \in E$, define $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$.

Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be any path from vertex v_0 to vertex v_k . Then p is a shortest path from v_0 to v_k with weight function w if and only if it is a shortest path with weight function $\hat{w}$. That is, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.

Preserving Shortest Paths by Reweighting

Lemma: Reweighting Does Not Change Shortest Paths

Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, let $h : V \rightarrow \mathbb{R}$ be any function mapping vertices to real numbers.

For each edge $(u, v) \in E$, define $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$.

Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be any path from vertex v_0 to vertex v_k . Then p is a shortest path from v_0 to v_k with weight function w if and only if it is a shortest path with weight function $\hat{w}$. That is, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.

Furthermore, G has a negative-weight cycle using weight function w if and only if G has a negative-weight cycle using weight function $\hat{w}$.

Proof of Reweighting Does Not Change Shortest Paths Lemma

Proof Part I

We start by showing that $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.

Proof of Reweighting Does Not Change Shortest Paths Lemma

Proof Part I

We start by showing that $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.

$$\hat{w}(p) = \sum_{i=1}^k \hat{w}(v_{i-1}, v_i) \tag{3}$$

$$= \sum_{i=1}^k (w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)) \tag{4}$$

$$= \left(\sum_{i=1}^k w(v_{i-1}, v_i) \right) + h(v_0) - h(v_k) \text{ (because the sum telescopes)} \tag{5}$$

$$= w(p) + h(v_0) - h(v_k). \tag{6}$$

Proof of Reweighting Does Not Change Shortest Paths Lemma Cont.

Proof Part II

- Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.

Proof of Reweighting Does Not Change Shortest Paths Lemma Cont.

Proof Part II

- Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.
- Because $h(v_0)$ and $h(v_k)$ do not depend on the path, if one path from v_0 to v_k is shorter than another using weight function w , then it is also shorter using $\hat{w}$.

Proof of Reweighting Does Not Change Shortest Paths Lemma Cont.

Proof Part II

- Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.
- Because $h(v_0)$ and $h(v_k)$ do not depend on the path, if one path from v_0 to v_k is shorter than another using weight function w , then it is also shorter using $\hat{w}$.
- Thus, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.

Proof of Reweighting Does Not Change Shortest Paths Lemma Cont.

Proof Part II

- Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.
- Because $h(v_0)$ and $h(v_k)$ do not depend on the path, if one path from v_0 to v_k is shorter than another using weight function w , then it is also shorter using $\hat{w}$.
- Thus, $w(p) = \delta(v_0, v_k)$ if and only $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.
- Finally, we show that G has a negative-weight cycle using weight function w if and only if G has a negative-weight cycle using weight function $\hat{w}$.

Proof of Reweighting Does Not Change Shortest Paths Lemma Cont.

Proof Part II

- Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.
- Because $h(v_0)$ and $h(v_k)$ do not depend on the path, if one path from v_0 to v_k is shorter than another using weight function w , then it is also shorter using $\hat{w}$.
- Thus, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.
- Finally, we show that G has a negative-weight cycle using weight function w if and only if G has a negative-weight cycle using weight function $\hat{w}$.
- Consider any cycle $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$.

Proof of Reweighting Does Not Change Shortest Paths Lemma Cont.

Proof Part II

- Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.
- Because $h(v_0)$ and $h(v_k)$ do not depend on the path, if one path from v_0 to v_k is shorter than another using weight function w , then it is also shorter using $\hat{w}$.
- Thus, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.
- Finally, we show that G has a negative-weight cycle using weight function w if and only if G has a negative-weight cycle using weight function $\hat{w}$.
- Consider any cycle $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$.
- We already proved that $\hat{w}(c) = w(c) + h(v_0) - h(v_k)$.

Proof of Reweighting Does Not Change Shortest Paths Lemma Cont.

Proof Part II

- Therefore, any path p from v_0 to v_k has $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.
- Because $h(v_0)$ and $h(v_k)$ do not depend on the path, if one path from v_0 to v_k is shorter than another using weight function w , then it is also shorter using $\hat{w}$.
- Thus, $w(p) = \delta(v_0, v_k)$ if and only if $\hat{w}(p) = \hat{\delta}(v_0, v_k)$.
- Finally, we show that G has a negative-weight cycle using weight function w if and only if G has a negative-weight cycle using weight function $\hat{w}$.
- Consider any cycle $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$.
- We already proved that $\hat{w}(c) = w(c) + h(v_0) - h(v_k)$.
- Since $v_0 = v_k$, we have $\hat{w}(c) = w(c)$, and thus c has negative weight using w if and only if it has negative weight using $\hat{w}$.



Producing Nonnegative Weights by Reweighting

Weights and Proof That They Are All Nonnegative

- Our next goal is to ensure that the second property of $\hat{w}$ holds: $\hat{w}(u, v)$ must be non-negative for all edges $(u, v) \in E$.

Producing Nonnegative Weights by Reweighting

Weights and Proof That They Are All Nonnegative

- Our next goal is to ensure that the second property of $\hat{w}$ holds: $\hat{w}(u, v)$ must be non-negative for all edges $(u, v) \in E$.
- Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, create $G'(V', E')$, where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.

Producing Nonnegative Weights by Reweighting

Weights and Proof That They Are All Nonnegative

- Our next goal is to ensure that the second property of $\hat{w}$ holds: $\hat{w}(u, v)$ must be non-negative for all edges $(u, v) \in E$.
- Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, create $G'(V', E')$, where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.
- To incorporate s , extend the weight function w so that $w(s, v) = 0$ for all $v \in V$.

Producing Nonnegative Weights by Reweighting

Weights and Proof That They Are All Nonnegative

- Our next goal is to ensure that the second property of $\hat{w}$ holds: $\hat{w}(u, v)$ must be non-negative for all edges $(u, v) \in E$.
- Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, create $G'(V', E')$, where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.
- To incorporate s , extend the weight function w so that $w(s, v) = 0$ for all $v \in V$.
- Since no edges enter s , no shortest paths in G' , other than those with source s , contain s .

Producing Nonnegative Weights by Reweighting

Weights and Proof That They Are All Nonnegative

- Our next goal is to ensure that the second property of $\hat{w}$ holds: $\hat{w}(u, v)$ must be non-negative for all edges $(u, v) \in E$.
- Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, create $G'(V', E')$, where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.
- To incorporate s , extend the weight function w so that $w(s, v) = 0$ for all $v \in V$.
- Since no edges enter s , no shortest paths in G' , other than those with source s , contain s .
- Moreover, G' has no negative-weight cycles if and only if G has no negative-weight cycles.

Producing Nonnegative Weights by Reweighting

Weights and Proof That They Are All Nonnegative

- Our next goal is to ensure that the second property of $\hat{w}$ holds: $\hat{w}(u, v)$ must be non-negative for all edges $(u, v) \in E$.
- Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, create $G'(V', E')$, where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.
- To incorporate s , extend the weight function w so that $w(s, v) = 0$ for all $v \in V$.
- Since no edges enter s , no shortest paths in G' , other than those with source s , contain s .
- Moreover, G' has no negative-weight cycles if and only if G has no negative-weight cycles.
- Now suppose that G and G' have no negative-weight cycles, and define the function

$$h(v) = \delta(s, v) \text{ for all } v \in V'.$$

Producing Nonnegative Weights by Reweighting

Weights and Proof That They Are All Nonnegative

- Our next goal is to ensure that the second property of $\hat{w}$ holds: $\hat{w}(u, v)$ must be non-negative for all edges $(u, v) \in E$.
- Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, create $G'(V', E')$, where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.
- To incorporate s , extend the weight function w so that $w(s, v) = 0$ for all $v \in V$.
- Since no edges enter s , no shortest paths in G' , other than those with source s , contain s .
- Moreover, G' has no negative-weight cycles if and only if G has no negative-weight cycles.
- Now suppose that G and G' have no negative-weight cycles, and define the function

$$h(v) = \delta(s, v) \text{ for all } v \in V'.$$

- By the triangle inequality, we have $h(v) \leq h(u) + w(u, v)$ for all edges $(u, v) \in E'$.

Producing Nonnegative Weights by Reweighting

Weights and Proof That They Are All Nonnegative

- Our next goal is to ensure that the second property of $\hat{w}$ holds: $\hat{w}(u, v)$ must be non-negative for all edges $(u, v) \in E$.
- Given a weighted, directed graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$, create $G'(V', E')$, where $V' = V \cup \{s\}$ for some new vertex $s \notin V$ and $E' = E \cup \{(s, v) : v \in V\}$.
- To incorporate s , extend the weight function w so that $w(s, v) = 0$ for all $v \in V$.
- Since no edges enter s , no shortest paths in G' , other than those with source s , contain s .
- Moreover, G' has no negative-weight cycles if and only if G has no negative-weight cycles.
- Now suppose that G and G' have no negative-weight cycles, and define the function

$$h(v) = \delta(s, v) \text{ for all } v \in V'.$$

- By the triangle inequality, we have $h(v) \leq h(u) + w(u, v)$ for all edges $(u, v) \in E'$.
- Thus, by defining reweighted edge weights $\hat{w}$ as described, we have

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0,$$

thereby satisfying the second property.

Pseudocode of the Johnson's Algorithm

JOHNSON(G, w)

```

1  compute  $G'$ , where  $G'.V = G.V \cup \{s\}$ ,
    $G'.E = G.E \cup \{(s, v) : v \in G.V\}$ , and
    $w(s, v) = 0$  for all  $v \in G.V$ 
2  if BELLMAN-FORD( $G', w, s$ ) == FALSE
3      print "the input graph contains a negative-weight cycle"
4  else for each vertex  $v \in G'.V$ 
5      set  $h(v)$  to the value of  $\delta(s, v)$ 
       computed by the Bellman-Ford algorithm
6  for each edge  $(u, v) \in G'.E$ 
7       $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ 
8  let  $D = (d_{uv})$  be a new  $n \times n$  matrix
9  for each vertex  $u \in G.V$ 
10     run DIJKSTRA( $G, \hat{w}, u$ ) to compute  $\hat{\delta}(u, v)$  for all  $v \in G.V$ 
11     for each vertex  $v \in G.V$ 
12          $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$ 
13  return  $D$ 

```

Figure: Running time: BELLMAN-FORD + $n \times$ Dijkstra.